

Хранилища в оперативной памяти

Table of contents

Хранилища в оперативной памяти	1
Аналитика: DuckDB и векторизация	1
Кэширование: Redis	2
Что в итоге	2

Хранилища в оперативной памяти

Колоночное хранение ускоряет аналитику за счёт физической раскладки данных: система читает меньше лишних байт. Есть другой способ специализации — убрать диск из горячего пути и держать рабочие данные в оперативной памяти.

RAM на порядки быстрее диска для случайного доступа. Это радикально меняет дизайн: не нужно минимизировать дисковые обращения, можно использовать более простые структуры (хеш-таблицы вместо B-tree), а оптимизация смещается к локальности данных, CPU-кешам и сетевым задержкам.

Лекция охватывает два класса систем, работающих в оперативной памяти: - **OLTP** (Redis, Memcached) — хранилища «ключ-значение» для кеша, сессий и ограничения частоты запросов. Простые структуры данных (хеш-таблицы, списки, множества), миллионы операций в секунду. - **OLAP** (DuckDB) — аналитическая база данных прямо в процессе приложения. Векторизованное выполнение (SIMD), колоночный формат в памяти, встроенная в Python/R/Node.js.

Базы в оперативной памяти критичны для сценариев с низкой латентностью: торги в рекламе в реальном времени (решение за менее 10 мс), финансовый трейдинг, игровые лидерборды. Также DuckDB сильно упростил анализ данных: вместо Pandas (медленный на больших данных) можно делать SQL-запросы прямо к Parquet-файлам с производительностью, близкой к ClickHouse.

Аналитика: DuckDB и векторизация

Векторизованное выполнение — обработка данных батчами (обычно 1–10 тыс. строк) вместо «строка-за-строкой»: - Один оператор работает сразу по вектору значений - SIMD/AVX инструкции CPU и меньшие ветвления - Лучшая локальность: кэш-линии и предвыборка памяти - Меньше интерпретационных накладных расходов и виртуальных вызовов

Почему это важно для аналитики: - Колоночное хранение держит данные одного типа подряд, удобно для векторов - Сжатие (RLE/Dictionary/Delta, описанные выше) распаковывается пачками и дает быстрые сканы - Поздняя материализация: фильтрация по колонкам до сборки строк

DuckDB — встраиваемая аналитическая СУБД с векторизованным движком:

```
duckdb.sql("SELECT * FROM 'data.parquet' WHERE price > 100")
```

Архитектура: - Встраивается в процесс (как SQLite), без сервера - Zero-copy чтение из Parquet/CSV/JSON напрямую - Морселизация: данные обрабатываются кусками ~1024 строки - Адаптивное выполнение запроса: план корректируется по статистике во время выполнения

Когда DuckDB vs альтернативы: - DuckDB: локальная аналитика на ноутбуке, 10 ГБ–1 ТБ данных, нужен SQL - ClickHouse: серверная OLAP, петабайты, распределённые запросы - Pandas: императивный код, < 10 ГБ, Python-экосистема

Где используют на практике: - BI-дашборды и Ad-hoc анализ (быстрые сканы больших таблиц) - ETL/ELT и проверка качества данных - Feature engineering в ML и аналитика экспериментов - Встраиваемая аналитика в приложениях и аналитических продуктах - Интерактивный анализ логов/метрик на ноутбуках и edge-узлах

Кэширование: Redis

Хранилище типа «ключ-значение» в оперативной памяти.

Архитектура: - Однопоточная модель + event-driven I/O (epoll/kqueue) - Нет накладных расходов на синхронизацию

Структуры данных: - **Skip List** (sorted sets) — упорядоченная структура в памяти; **ZADD leaderboard 100 "player1"** - **Incremental Rehashing** — постепенное перехэширование без блокировки - **HyperLogLog** — вероятностный подсчёт уникальных; 12 КБ для миллиардов элементов - Strings, Lists, Sets, Hashes, Bitmaps, Streams

Применение: кэширование, сессии, очереди, аналитика в реальном времени

Что в итоге

Хранилища в памяти выигрывают не новым типом запроса, а тем, что убирают самый дорогой уровень задержки из горячего пути. Это хорошо работает для кешей, сессий, счётчиков, лидербордов и локальной аналитики.

Но память не отменяет задачу поиска. Если запрос не сводится к точному ключу, всё равно нужна структура, которая заранее сузит кандидатов: по словам, префиксам, категориям, координатам или близости. Дальше переходим к таким специализированным индексам.